Artificial Intelligence and Robotics

A Complete Course in Plain English

ARTIFICIAL INTELLIGENCE AND ROBOTICS

Volume Five

Natural Language Processing

TBUGE MECHATRONIX

Artificial Intelligence and Robotics, Volume Five: Natural Language Processing

First edition. TBUGE Mechatronix.

This volume treats language as evidence rather than magic. It counts words before it embeds them, builds a language model small enough to compute on paper, and measures every claim on text the model has not seen.

The programme runs to forty-eight volumes. Volume Four gave the machine eyes; this one gives it the written word and the means to be held to account for what it says about it.

Course codes have been left out on purpose. A subject should be named, not numbered.

Contents

Preface **6**

How This Volume Is Arranged **7**

Part One · Language as Data **9**

What Language Looks Like to a Machine 10

Cleaning, Splitting and Counting 15

The Statistics of Words 21

Part Two · Meaning Without Understanding **26**

Words as Vectors 27

The Chain of Small Probabilities 33

Smoothing, and the Cost of the Unseen 38

Part Three · Learning From Text **42**

Context Windows and Learned Predictions 43

Classifying Text 48

Extraction: Names, Relations and Structure 52

Search, Retrieval and Answering Questions 58

Part Four · Language in the World **64**

Translation and the Problem of Equivalence 65

Speech: Sound Into Words 70

Evaluation, Bias and the Decision to Ship 76

Glossary **81**

Sources and Further Reading **84**

Preface

Language is the easiest domain in artificial intelligence to demonstrate and the hardest to evaluate. A system that produces three fluent paragraphs looks finished; a system that classifies ten thousand support tickets looks dull. The fluent one may be wrong in ways nobody can measure, and the dull one may be right in ways that pay for themselves in a month.

This volume is written from the dull end. It spends its first hundred pages counting words, because the counting is where the difficulty hides: how text is cut into pieces, how often words repeat, how much of a language is a small number of very common tokens and an enormous tail of rare ones. It builds a language model small enough to compute with a pencil, so that the idea of a probability over sentences stops being abstract. Only then does it turn to vectors, networks, translation and speech.

The later chapters are about the part of the subject that is currently changing fastest and therefore dates fastest. The methods will be superseded. What will not be superseded is the discipline of asking what a score measures, on which text, split how, and compared with what baseline. That discipline is the actual content of the volume, and it is why the chapters on evaluation come before the chapter that tells you whether to ship.

What you need

A text file, a terminal, and arithmetic. No calculus, no linear algebra beyond a dot product, no framework. Where a model appears — a bigram table, a classifier, a network — it is described in the smallest version that still shows the mechanism, and the numbers are small enough to check by hand.

What you will not find

Benchmarks quoted without their conditions. A single "accuracy" for a whole language. Claims about what a model understands. These are not omissions of modesty; they are the specific things this field gets wrong most often, and a book that repeated them would be teaching the mistakes.

A note on the examples

The text used in the worked examples is invented, short, and deliberately unremarkable. That is the point: a corpus of eight sentences is enough to show that an unseen bigram has probability zero, and a reader can verify every number on the page. Real corpora make the arithmetic expensive and the lesson no clearer.

The promise

By the last chapter you should be able to take a piece of text, a task, and a claim about performance, and answer three questions without hesitation: what exactly was counted, what was held out, and what would happen on the text that arrives next month.

How This Volume Is Arranged

Part One — Language as Data takes the machine's view. Text arrives as characters, is cut into tokens by rules that are choices rather than discoveries, and turns out to be wildly uneven: a handful of words account for most of the tokens and most of the vocabulary accounts for almost nothing. Counting is the first skill, and the chapters here do it on small, checkable examples.

Part Two — Meaning Without Understanding builds the classical statistical apparatus: words as vectors derived from the company they keep, sequences as chains of small probabilities, and the arithmetic of the unseen. The last of these is the technical heart of the volume, because every language model that has ever been built is, at bottom, a strategy for assigning a small probability to something it has never seen.

Part Three — Learning From Text moves to models trained on data: the context window, text classification, extraction, and retrieval. This is where the discipline of splitting and evaluating becomes decisive, since a text dataset can be contaminated in ways a numerical one cannot.

Part Four — Language in the World is about the applications that people actually buy: translation, speech interfaces, and the decision to put a language system in front of a user. The closing chapter is a checklist and an argument, and it is the one to read twice.

The chapters vary in shape on purpose. One is built around a table of counts, one around a worked translation, one around a failure in a call centre. A uniform template would make the book easier to write and worse to read.

Every chapter ends with work, under a heading that suits the work: Problems, Exercises, Working Notes, Things to Try.

PART ONE

Language as Data

CHAPTER ONE

What Language Looks Like to a Machine

The complaint came from a warehouse in Leeds. A spell-checking step in an order-entry system had begun flagging every product code that contained the letters "BOLT", because a dictionary of ordinary English does not contain that string, and the nearest word by edit distance was "BOAT". The system was not malfunctioning. It was doing exactly what it had been told: compare each token with a list of known words, and flag the ones that are close to a known word but not equal to it.

The interesting part is the complaint that followed. The warehouse asked for the dictionary to be updated. The engineer, sensibly, asked instead why product codes were being spell-checked at all, and discovered that the pipeline ran every field through the same cleaning function because the function was convenient.

This is the pattern of the first part of this book. Language processing fails less often because a model is weak than because a decision about *what a word is* was made once, by accident, and then applied to everything.

A String Is Not a Word

At the lowest level, text is a sequence of code points encoded as bytes. UTF-8 uses one byte for ordinary English letters, two for accented Latin characters, three for most other scripts, four for emoji and rarer symbols.

Text	Code points	Bytes in UTF-8
cat	3	3
café	4	5
€	1	3
—	1	3

Three consequences follow, and each has produced a real bug in a real system.

Length is not the number of visible characters. A field limited to forty bytes holds thirteen emoji, not forty.

Upper and lower case are not always paired. The German sharp s, the Turkish dotless i, and the Greek final sigma all break the assumption that case conversion is reversible. Turkish in particular has produced a generation of systems that turn a capital I into a dotted i when the correct answer is the dotless form. Emoji, which take four bytes each, make the same point about length.

Slicing by byte can cut a character in half. A truncation at two hundred bytes may land inside a three-byte character, producing invalid text that later stages will reject or, worse, silently corrupt.

None of this is exotic. It is the everyday arithmetic of handling text, and it is the reason production systems agree on an encoding — UTF-8 — at every boundary and validate it on entry.

Ambiguity Is Not an Edge Case

A word in isolation carries almost no information. "Bank" is a financial institution or a river edge. "Sanction" means to permit and to punish. "Fast" means quick and immobile. "Bolt" is a fastener, a verb for running, and a roll of cloth.

Human readers resolve these instantly using context, and the resolution feels like knowledge of meaning. For a machine, the resolution is a statistical decision over neighbouring words, and it is worth seeing that stated plainly, because it sets the ceiling of everything in this volume.

Consider the two sentences:

> The bank raised its rate. The bank was steep and muddy.

A classifier that sees only the word "bank" cannot do better than guess. One that sees "rate" or "muddy" can do very well. The information that disambiguates is *distributional* — it lives in which words appear near which others — and that single observation is the foundation of the modern field.

Cutting Sentences

Before words, sentences. It sounds trivial until the text contains "Dr. Smith arrived at 4.30 p.m. on Tuesday," which has three apparent sentence ends and one actual one.

Rules of thumb that work reasonably: a full stop is a sentence end if followed by a space and a capital letter; abbreviations are a known list; a decimal point is never a sentence end; a single capital letter followed by a period is usually an initial.

The failure cases matter in production. Academic citations, legal numbering, and chemical names produce sentences that end nowhere and paragraphs that end mid-clause. When a downstream task depends on sentence boundaries — summarisation, or anything that scores per sentence — the boundary errors propagate invisibly, and a model can appear to fail at summarisation when it is failing at splitting.

Distance Between Strings

One of the oldest and most useful tools treats text as a sequence of characters and asks how far apart two strings are. The **edit distance** is the smallest number of insertions, deletions and substitutions that turn one string into another.

Take "kitten" and "sitting". Substitute k for s at the first position, substitute e for i at the fifth, insert g at the end. Three edits, so the distance is three.

The recurrence is a small table, and it is worth filling once by hand:

```
distance[i][j] = min(distance[i-1][j] + 1, distance[i][j-1] + 1, distance[i-1][j-1] + cost)
```

where cost is zero if the characters match and one if they do not. The table has one more row and column than the strings, filled from the top left, and the answer is in the bottom right corner.

This distance is the engine behind spelling correction, fuzzy matching of names, and the detection of near-duplicate records — the last of which is more valuable than it sounds, because duplicate near-matches are how a text dataset gets contaminated.

What a Pipeline Looks Like

Four stages, and every system in this volume is some arrangement of them.

Stage	Question	Typical decisions
Acquisition	What text do we have, and from where?	Source, licence, time span, language
Normalisation	What counts as the same text?	Case, accents, punctuation, numbers, encodings
Tokenisation	What are the units?	Words, subwords, characters, sentences
Representation	How is a unit turned into numbers?	Counts, vectors, identifiers

Each decision is a modelling choice with a consequence, and the consequences are not symmetric. Lowercasing makes "Bank" and "bank" identical, which helps topic models and hurts named-entity recognition. Removing punctuation makes "3.5" into "35", which is harmless for sentiment and catastrophic for finance. Stripping stop words removes "not", which reverses the meaning of every sentence that contains it.

The rule that survives every project: decide once, write the decision down, and apply the same function to training text and to production text. A pipeline whose training data was cleaned differently from its live data is measuring one thing and deploying another.

Working Notes

1. How many bytes does the string "naïve café" occupy in UTF-8? How many characters does a human see?

2. Compute the edit distance between "form" and "farm", and between "form" and "from". Which pair would a spell-checker confuse more readily, and why?

3. A system truncates user input at 500 bytes. Give two inputs where this produces a different result from truncating at 500 characters.

4. "I did not like the film" and "I did like the film" differ by one token. Explain what a stop-word filter does to the second sentence and to a sentiment classifier's accuracy.

5. A colleague proposes lowercasing all text, removing punctuation, and stripping digits before training a classifier that must detect invoice totals. Name two specific failures.

6. List the four pipeline stages for a system that reads customer emails and routes them to a department. Write one decision for each stage.

CHAPTER TWO

Cleaning, Splitting and Counting

A sentiment model was trained on twenty thousand customer emails and performed beautifully in testing. In production it developed a curious habit: any message ending with a formal sign-off was scored as negative, however cheerful the body.

The cause took an afternoon to find. The most predictive phrase in the entire training set was not a phrase about the product at all. It was the boilerplate signature that the company's own complaint-handling team appended to every message it sent, and the training data had been assembled from threads in which those appended messages were included. The model had learned, with perfect statistical justification, that this signature predicted dissatisfaction — because in that dataset, it did.

This chapter is about the decisions that come before any model: what to keep, what to cut, and what a token is. They are unglamorous and they determine more of the outcome than the choice of algorithm.

Cleaning Is a Claim About Meaning

Every cleaning step asserts that two different strings mean the same thing. That assertion is sometimes true, and the cases where it is false are where the bugs live.

Lowercasing asserts that case is irrelevant. True for topic modelling, false for names ("Apple" the company, "apple" the fruit), for German nouns, and for any text where shouting is a signal.

Removing punctuation asserts that it carries no information. False for "3.5", for "U.S.", for "don't", and for emoticons.

Removing stop words asserts that frequent words are empty. False for negation, for questions ("what", "why"), and for anything where the distinction between "the report" and "a report" matters.

Stemming asserts that "running", "runs" and "ran" are one thing. True for retrieval, false for grammar-sensitive tasks, and always approximate — the classic stemmer output "gener" for "general", "generous" and "generation".

Removing HTML and boilerplate asserts that markup is not content. Almost always true, and almost always done incompletely, which is how the opening story happened.

Decision	Helps	Hurts
Lowercase	Vocabulary size, recall	Names, emphasis, German
Strip punctuation	Simplicity	Decimals, abbreviations, contractions
Remove stop words	Speed, topic clarity	Negation, questions, style
Stem or lemmatise	Grouping variants	Precision, rare terms
Strip digits	Noise reduction	Quantities, dates, codes

The general principle: clean for a task, not for a corpus. If you cannot say which errors the cleaning prevents and which it introduces, it is a habit rather than a decision.

Where Whitespace Fails

Splitting on spaces gets you a long way in English and then stops.

> "I don't think it's working" becomes: I | don't | think | it's | working

That is five tokens, and "don't" carries a negation that is invisible to a model that only sees the string. Splitting contractions gives "do" and "n't", which makes the negation explicit. Whether that is better depends on the task, and either choice must be applied consistently.

Other cases where spaces are the wrong boundary:

Hyphenated compounds ("state-of-the-art") — one token or five?

Numbers with units ("12mm", "12 mm", "12.0 mm") — three forms of one fact.

URLs and file paths — full of punctuation that a naive splitter shreds.

Languages without spaces — Chinese and Japanese require segmentation of their own, and a whitespace splitter returns one enormous token per sentence.

Social media text — hashtags, mentions, repeated letters for emphasis.

Subwords, and the Vocabulary Problem

The tension is simple. A word-level vocabulary must include every word the system will ever meet, and English has an unbounded supply: product names, surnames, compounds, typos, and the plurals and tenses of all of them. A vocabulary of a million words still misses something every day. An unknown word must be mapped to a single placeholder, and that placeholder means nothing.

The modern answer is to stop treating words as atoms and learn **subword** units: pieces of words that recur often enough to be worth having.

The standard algorithm is byte-pair encoding, which is a counting procedure and can be run by hand on a toy corpus.

Start with these words and their frequencies: low (5), lower (2), newest (6), widest (3). Split each into characters, then count adjacent pairs across the whole corpus.

Pair	Count
e s	9
s t	9
w e	8
e w	6
n e	6

The most frequent pair is "es" (the tie with "st" is broken arbitrarily). Merge it everywhere it occurs: "newest" becomes n e w es t, "widest" becomes w i d es t, and the pair counts are recomputed. Repeat. After a few dozen merges on a real corpus, common word-pieces have appeared: "ing", "tion", "pre", "est".

The result is a vocabulary of thirty thousand to fifty thousand pieces that can encode any string whatsoever, because the fallback is characters. Unknown words disappear as a category; an unfamiliar product name becomes two or three familiar fragments.

The cost is that token counts inflate. English text averages roughly one and a third tokens per word, so a thousand-word document becomes about 1,300 tokens. Since nearly every modern language system bills, or times, or limits by token, that inflation is not academic:

Document	Words	Tokens (approx.)	Effect
Email	250	330	Fits easily
Report	4,000	5,300	Approaches context limits
Manual	60,000	80,000	Must be chunked

There is a second cost that is easier to miss. Tokenisation depends on the tokeniser's training corpus, so two systems with different tokenisers cannot compare token counts, and a per-token price is not a per-word price.

Tokens and Types

Two counts, and confusing them is a classic error.

The **token count** is the number of running instances: "the cat sat on the mat" has six tokens.

The **type count** is the number of distinct forms: "the", "cat", "sat", "on", "mat" — five types, because "the" appears twice.

A useful corpus statistic: a million tokens of English news text contains roughly forty thousand types, and about half of those types occur exactly once. That long tail of singletons is the central difficulty of the subject, and the next chapter is about what it does to any model built on counting.

Numbers, Dates and Units

Nothing in ordinary tokenisation handles quantities well, and quantities are where text processing meets consequences.

"1,200" tokenises as "1", ",", "200". "12.5 mm" becomes "12", ".", "5", "mm". "£1.2m" becomes a currency symbol, a decimal and a letter. None of these is wrong at the level of tokens, and all of them are wrong at the level of meaning: the value is distributed across pieces, and any downstream task that reads single tokens has lost it.

The repair is a normalisation step that recognises quantities as units and rewrites them in a canonical form before tokenisation:

Input	Canonical	Note
1,200	1200	Thousands separator removed
1.200,50	1200.50	European convention

Input	Canonical	Note
£1.2m	1200000 GBP	Multiplier expanded
12.5mm	12.5 mm	Space inserted for splitting
3 May 2025	2025-05-03	Ambiguity removed

Two warnings about this table.

The European convention is a genuine hazard. "1.200" is one thousand two hundred in Germany and one point two in Britain, and a pipeline that guesses wrongly is off by a factor of a thousand. Guess from the document, not from the number: if other figures in the same document use a comma as a decimal separator, that is the convention.

Date normalisation can invert day and month. "03/05/2024" is the third of May in Britain and the fifth of March in the United States, and there is no way to tell from the string. Systems that must handle both should refuse to normalise ambiguous dates silently, and should keep the original text alongside the parsed value so that a human can check it.

The reason this belongs in a chapter about tokenisation rather than in a chapter about numbers is that the decision has to be made before tokens exist. Once "1,200" has become three tokens, the information that it was a single quantity is gone, and the model must rediscover it from context — which, for numbers, it does badly.

A Worked Count

Take this fragment, three sentences, and count it.

> The pump failed again. The pump was replaced. The new pump failed.

Tokens: 6 + 4 + 4 = 14. Types: the, pump, failed, again, was, replaced, new = 7.

"pump" occurs 3 times, "the" 3 times, "failed" twice. The two most frequent words account for 6 of 14 tokens, or 43%. That proportion is absurd for three sentences and roughly right for a corpus: frequency is concentrated, and it is concentrated in words that carry little content.

Exercises

1. Tokenise this sentence by whitespace and then by a rule that splits contractions: "It's not the same, is it?" How many tokens in each case?

2. A corpus contains 500,000 tokens and 31,000 types. What is the ratio of types to tokens, and what does a low ratio suggest about the text?

3. Compute the two most frequent adjacent character pairs in: "testing", "tester", "resting", "rested", weighting each word once.

4. A model's context window holds 8,000 tokens. How many words of ordinary English does that correspond to, at 1.3 tokens per word? How does the answer change for a language with longer words?

5. A pipeline removes stop words before classification. Construct a twosentence example where this changes the label, and one where it does not.

6. An email dataset includes quoted replies from previous messages. Describe two ways this contaminates a sentiment model and how you would detect it.

CHAPTER THREE

The Statistics of Words

The classifier worked on everything except the one thing it was built for. It routed emails to six departments with an accuracy of 91%, and it failed, consistently, on messages about a single component: the model XYZ-40 seal kit.

The reason was visible in a frequency table. Across two years of email, "seal" appeared 4,100 times, "kit" 2,900, "leak" 1,700 — and "XYZ-40" appeared eleven times. Eleven examples cannot teach a model anything, and the model had done the only sensible thing available to it: it had learned that the rare token was noise and ignored it.

This chapter is about the shape of that distribution. It is the same shape in every language, in every corpus, and it explains a large fraction of the failures in applied language processing.

A Frequency Table Worth Memorising

Take a corpus of one million tokens of ordinary English and rank the words by frequency. The counts fall roughly as the reciprocal of the rank:

Rank	Approximate count	Share of tokens
1	60,000	6%
2	30,000	3%
5	12,000	1.2%
10	6,000	0.6%
100	600	0.06%
1,000	60	0.006%

This is **Zipf's law**, and it says that a word's frequency is inversely proportional to its rank. It is not an exact law — the very top is flatter than the formula predicts, and the far tail is steeper — but it is close enough to be used for plan-

ning.

Two consequences, both of which decide projects.

The first is coverage. A small number of words accounts for most of the running text:

Vocabulary size	Share of tokens covered
Top 10 words	about 25%
Top 100 words	about 50%
Top 1,000 words	about 70%
Top 10,000 words	about 90%

So a system that handles the ten thousand most common words is missing one token in ten, and the missing tenth contains almost all the information. "The pump was replaced" is mostly common words; "pump" is not in the first hundred, and the thing that distinguishes a failure report from a delivery query is nearly always in the tail.

The second consequence is about vocabulary growth. As a corpus grows, the number of distinct types grows too, but sublinearly. The empirical relation is **Heaps' law**:

> types = k x tokens^0.5 (approximately, with the exponent usually between 0.4 and 0.6)

Doubling the corpus multiplies the vocabulary by about 1.4. Which means: no corpus is ever complete, and there is always a next word you have never seen. Every model in the rest of this book has to have an answer for that word, and the answers are not equivalent.

What Frequency Does Not Tell You

Frequency is not importance. "The" is the most frequent token in English and conveys almost nothing; a part number occurring eleven times may be the entire point.

Frequency also does not identify phrases. "New York" is a unit of meaning, and so is "strong tea", and neither is discoverable from the counts of the individual words. What identifies them is that the words co-occur far more often than chance would predict.

That comparison has a standard measure, the **pointwise mutual information**:

PMI(a, b) = log2 [P(a, b) / (P(a) x P(b))]

Worked, on the million-token corpus. Suppose "strong" occurs 2,000 times, "tea" 1,000 times, and the pair "strong tea" occurs 30 times as an adjacent pair.

P(strong, tea) = 30 / 1,000,000 = 0.00003 P(strong) x P(tea) = 0.002 x 0.001 = 0.000002 ratio = 15 PMI = log2 15 ~ 3.9

A PMI near zero means the words appear together as often as independent chance predicts — which is what happens with "the" and almost anything, because "the" is everywhere. A high PMI means genuine association, and the list of high-PMI pairs in any corpus is a surprisingly good dictionary of the domain's terminology.

The measure has a known pathology: it over-rewards rare pairs. A pair of words that occur together twice and never apart gets an enormous PMI. Practical implementations therefore require a minimum count, and often use a variant that discounts rare events.

Pair	Count	PMI	Reading
strong tea	30	3.9	Real collocation
of the	21,000	near 0	Grammar, not meaning
XYZ-40 seal	9	high	Domain term, few examples
good bad	40	low	Antonyms, rarely adjacent

The Rare Class Is the Task

Put the two facts together — coverage concentrated in common words, and the informative words in the tail — and the practical difficulty of applied language processing is stated: **the thing you must detect is almost always rare**.

A defect description, a fraud signal, a rare diagnosis, an unusual product complaint: all of them live in the tail, where counts are small and models are uncertain. This is why the previous volume spent so long on class imbalance, and why the numbers look the same here.

The responses are also the same, and they are worth listing because they are the ones that work.

Get more examples of the rare thing. Nothing else substitutes.

Represent words as pieces rather than atoms, so that a rare word shares fragments with common ones.

Use context, not the word alone. "XYZ-40" is rare, but "leak", "seal" and "replaced" are not, and a model that reads windows rather than words can use them.

Choose metrics that do not reward ignoring the rare class, which is to say anything except accuracy.

A Counting Exercise, Done Fully

Four short documents, and a question: which word is most characteristic of this little collection?

d1: pump seal leak d2: pump seal replaced d3: pump noisy d4: valve leak

Tokens: 11. Types: pump, seal, leak, replaced, noisy, valve. Counts: pump 3, seal 2, leak 2, replaced 1, noisy 1, valve 1.

Frequency says "pump". But "pump" appears in three of four documents, and "leak" in two — so a document-level measure would rank pump first and leak second.

Now compute a document-frequency-weighted score: multiply the count of a word in a document by the inverse of how many documents contain that word. "leak" has document frequency 2, so its weight is higher than "pump"'s document frequency of 3. This is the intuition behind tf-idf, and it is the reason a search engine does not simply return documents containing the most common term.

The arithmetic for "leak" in d1: term frequency 1, inverse document frequency log(4/2) = 0.69, score 0.69. For "pump" in d1: term frequency 1, log(4/3) = 0.29, score 0.29. The rarer word wins, which is the correct answer for retrieval, and it is a purely statistical result — no understanding of pumps or leaks was required.

Problems

1. In a two-million-token corpus, rank 1 occurs 110,000 times. Using Zipf's law, estimate the count at ranks 2, 10 and 500.

2. A corpus of 200,000 tokens has 18,000 types. If it is doubled, estimate the number of types using Heaps' law with an exponent of 0.5.

3. Compute PMI for a pair occurring 50 times, where each word occurs 5,000 times, in a corpus of 2,000,000 tokens. Is it a collocation?

4. Explain, with the coverage table, why a keyword list of 200 words can classify some support tickets well and fail on others.

5. A domain has 900,000 tokens and a product name that appears 14 times. Propose three ways to make a classifier sensitive to it.

6. Two corpora, one of news and one of legal contracts, have the same number of tokens. Which has more types, and why? What does that imply for a shared vocabulary?

PART TWO

Meaning Without Understanding

CHAPTER FOUR

Words as Vectors

A retrieval system was asked for documents about "refunds" and returned nothing useful, although the archive contained four hundred pages on the subject. Every one of them used the word "credit note".

A human archivist would have found them in a minute, because a credit note and a refund are the same transaction described by two names. The system could not, because the two names share no letters, and a keyword search is a test of shared letters.

The fix required a representation in which "refund" and "credit note" are near each other. This chapter is about how such a representation is built, what its distances mean, and where the whole idea quietly fails.

The Distributional Idea

The assumption is old, and it was stated most compactly by J. R. Firth: you shall know a word by the company it keeps.

If two words appear in similar contexts, they are used similarly, and — for most practical purposes — they mean similar things. "Refund" appears near "request", "customer", "issued", "within fourteen days". So does "credit note". The contexts are nearly the same, so the words should be near each other in whatever space we construct.

The construction is mechanical. For each word, count which other words appear near it across a corpus; treat that vector of counts as the word's representation; then compare vectors by angle.

A Small Co-occurrence Table

Three words and their counts against three context words, in a tiny corpus:

Word	customer	days	issued
refund	20	15	10
credit	18	14	12
valve	2	0	1

"Refund" and "credit" have similar profiles; "valve" does not. That similarity is what a vector space formalises.

Cosine Similarity, Worked

Comparing vectors by angle rather than by distance has one great advantage: it ignores length. A word appearing fifty times more often than another will have larger counts everywhere, and angle removes that scale.

cosine = (a . b) / (|a| x |b|)

Take a = (1, 2, 2) and b = (2, 1, 2).

dot = 1x2 + 2x1 + 2x2 = 2 + 2 + 4 = 8 |a| = sqrt(1 + 4 + 4) = 3 |b| = sqrt(4 + 1 + 4) = 3 cosine = 8 / 9 = 0.889

An angle of about 27 degrees. In practice, trained word vectors produce cosine values in a narrow band — most pairs sit between 0 and 0.4 — so a value of 0.889 is not "similarity of 89%"; it is "unusually close".

Cosine	Typical reading
0.85 and above	Near-synonyms or the same entity
0.6 to 0.85	Same topic or domain
0.4 to 0.6	Related, not interchangeable
Below 0.3	Uninformative

These bands are conventions, not measurements, and they shift with the model. They are useful for sanity checks, dangerous as thresholds.

How the Vectors Are Learned

The hand-built table above is what early systems used, and it works badly: it is enormous, mostly zeros, and dominated by frequent words.

The modern method trains the vectors as a by-product of a prediction task. For each word in a corpus, take its neighbours as a target, and adjust the vector so that it predicts them better. Words that occur in similar company end up with similar vectors because they need to make the same predictions. The vectors are then a few hundred numbers per word — three hundred is conventional — and they are dense rather than sparse.

The arithmetic of analogy falls out of this. Because the training forces consistent directions onto consistent relationships, vector arithmetic approximates meaning arithmetic:

```
king - man + woman ~ queen
```

It works often enough to be striking and often enough to be wrong that it has generated a decade of overclaiming. The failures are usually in domains where the corpus itself is inconsistent, and they are always about the corpus, never about the geometry.

One practical number: an embedding table for 100,000 words at 300 dimensions, stored as 4-byte numbers, is 120 megabytes. That is why embedding tables are shipped as files, versioned, and why a mismatch between the table used in training and the one used at inference produces predictions that are wrong in a way no error message will reveal.

Search at Scale

Once everything is a vector, finding similar things is a geometric question: which points are nearest to this one? The naive answer is to compare against all of them, and the arithmetic of that is worth doing because it decides the architecture.

Take 100,000 documents represented as 300-dimensional vectors of 4-byte numbers.

```
storage = 100,000 x 300 x 4 bytes = 120 megabytes cost per query = 100,000 x 300 = 30 million multiply-additions
```

Thirty million operations is nothing for a modern processor — a few milliseconds — so a single query against a hundred thousand documents is fine by brute force. At a hundred queries per second it is three billion operations per second, which is no longer nothing, and at a million documents it is ten times

worse again.

The standard remedy is an approximate index. Instead of comparing with everything, the index groups vectors into clusters and searches only the nearby ones, or arranges them in a graph that can be walked towards the query. The result is a search that examines a small fraction of the collection and returns almost the same answer.

"Almost" is the part that requires measurement. An approximate index has a **recall**: the fraction of the true nearest neighbours it actually returns. A well-tuned index returns ninety-five per cent of the true top ten, at perhaps a fiftieth of the cost of exhaustive search.

Index setting	Recall at 10	Relative speed
Exhaustive	100%	1x
High accuracy	98%	8x
Balanced	95%	40x
Fast	80%	200x

The last row is a trap that appears in real systems: retrieval gets faster, the answers get slightly worse, and because nothing in the pipeline reports recall, the degradation is invisible until someone notices that the assistant no longer mentions the obvious document.

The measurement that prevents this is small. Take two hundred real queries, compute the exhaustive answer for each, and compare with what the index returns. That is the index's recall, and it belongs in the monitoring dashboard alongside latency.

What the Distances Do Not Mean

Four cautions, each of which has cost a project.

Antonyms are close. "Good" and "bad" appear in nearly identical contexts, and their vectors reflect that. A similarity search for "good" will return "bad" cheerfully.

Similarity is not substitutability. "Refund" and "credit note" are near neighbours and cannot be swapped in a sentence. Retrieval benefits; generation suffers.

Rare words get bad vectors. A word seen eleven times has eleven contexts, and its vector is noise with a direction. This is the same tail problem as the previous chapter, wearing different clothes.

A word is not one thing. Before contextual models, "bank" had a single vector, an average of its financial and riverside uses. Averaging two meanings produces a point between them, which is near neither.

The last point is why modern systems compute a *contextual* representation: the vector for a word depends on the sentence it is in. The word "bank" gets one vector in "raised its rate" and another in "steep and muddy". Mechanically this is the same idea — position in a space determined by company — applied per occurrence rather than per word.

Where This Is Useful

Three applications, in order of how well they work.

Retrieval and clustering are the natural home. Finding documents that discuss the same subject, grouping complaints into themes, deduplicating a corpus — all of these tolerate approximate similarity and benefit enormously from synonym matching.

Classification features work well when data is plentiful and classes are coarse. A classifier fed averaged word vectors on ten thousand labelled emails will beat a keyword list comfortably.

Generation and rewriting are where the caution belongs. Nearest neighbours are not paraphrases, and a system that replaces words with their nearest neighbours produces text that is grammatically fine and semantically wrong — the failure mode that makes machine translation output read as though it were written by someone who had learned the language from a dictionary.

Things to Try

Take two hundred sentences from a domain you know, and build the cheapest possible co-occurrence table: for each word, count the words appearing within two positions, keeping only the two hundred most frequent. Compute cosine similarity for a handful of pairs you have opinions about — "leak" and "drip", "refund" and "return", "urgent" and "important". The numbers will be less impressive than the ones in the papers and more informative, because you will know which of them are wrong.

Then find the nearest neighbours of a word that has two meanings in your domain, and see which meaning dominates. That single experiment explains why contextual models exist.

CHAPTER FIVE

The Chain of Small Probabilities

The autocomplete in a ticketing system had one charming defect. After an agent typed "kind", it offered "regards" with a confidence of 0.94, whatever the sentence. "Kind regards" was the commonest pair in the corpus by a wide margin, and the model was simply reporting the truth about the corpus.

The defect was not statistical. It was that the model had no way to represent "kind of urgent" or "kindly note", because it looked exactly one word back. Everything about a language model's competence, and everything about its silliness, follows from how far back it looks.

The Assumption

A language model assigns a probability to a sequence of words. Written out honestly, that is

$$P(w1, w2, ..., wn) = P(w1) \times P(w2 \mid w1) \times P(w3 \mid w1, w2) \times ...$$

and the conditioning context grows without limit. The **Markov assumption** truncates it: assume the next word depends only on the previous k words. With k = 1, a **bigram** model:

$$P(w1 ... wn) \sim P(w1) \times P(w2 \mid w1) \times P(w3 \mid w2) \times ...$$

This is false. It is also the foundation of a great deal of working technology, and its falseness is worth understanding precisely: the model is not claiming language is a Markov chain; it is claiming that the previous word carries most of the useful short-range information, and that the rest is too sparse to estimate.

Bigrams by Hand

Two sentences:

the cat sat on the mat the cat ate the fish

Add a boundary marker at the start and end of each, giving 14 tokens including boundaries. Count the pairs:

Pair	Count
the cat	2
cat sat	1
cat ate	1
sat on	1
on the	1
the mat	1
ate the	1
the fish	1

"The" begins four pairs in total, so conditional probabilities from "the" are:

P(cat | the) = 2/4 = 0.50 P(mat | the) = 1/4 = 0.25 P(fish | the) = 1/4 = 0.25

From "cat", two pairs begin, so each continuation has probability 0.5. The whole sentence "the cat sat on the mat" then has probability

1.00 x 0.50 x 0.50 x 1.00 x 1.00 x 0.25 = 0.0625

Every conditional in that product came from a table of eight entries. That is the entire mechanism: a language model is a lookup table with an opinion about what has not been seen.

Sparsity, Which Is the Real Subject

The table above is small enough to memorise. Now consider the actual numbers.

A corpus of a million tokens contains 40,000 distinct words and about 600,000 distinct bigrams. The number of possible bigrams is $40{,}000^2 = 1.6$ billion. So the corpus covers about 0.04% of the possible pairs, and the rest have no count at all.

A trigram model is worse: $40{,}000^3 = 6.4 \times 10^{13}$ possibilities, against perhaps 10 million observed trigrams. And the count of counts is brutal — most tri-

grams that occur, occur once.

Order	Context used	Distinct n-grams in a million tokens	Coverage of possibilities
1 (unigram)	none	40,000	0.1% of vocabulary seen
2 (bigram)	one word	600,000	0.04%
3 (trigram)	two words	8,000,000	0.00001%

The practical effect: an eight-word sentence using ordinary English has a decent chance of containing at least one pair that never appeared in the corpus. Under a raw bigram model, that sentence has probability zero — not small, zero. And because probabilities are multiplied, one zero annihilates the whole product, which is why the next chapter is devoted entirely to the arithmetic of the unseen.

One More Word, and the Data Doubles

It is tempting to conclude that a trigram model must be better than a bigram model, since it uses more context. In principle it is. In practice the extra context costs data at a rate that surprises people.

The reason is in the counts. A bigram model needs to have seen each pair often enough to estimate a probability; a trigram model needs each triple. For English text, the number of distinct trigrams in a corpus is roughly ten times the number of distinct bigrams, and most of them occur once. To estimate trigram probabilities as well as bigram probabilities were estimated, a corpus needs something like ten times the tokens — and the next order needs ten times more again.

There is a standard way to see the effect, and it is the table of counts by frequency:

Frequency in corpus	Bigrams	Trigrams
1 occurrence	68%	86%
2 occurrences	15%	8%
3 or more	17%	6%

Two thirds of observed bigrams were seen once; six sevenths of trigrams were. A count of one is not an estimate of anything — it is an anecdote — and

the smoothing methods of the next chapter exist precisely to handle this distribution of anecdotes.

The practical consequence for anyone building a system: adding context is not free, and the benefit must be measured on held-out text rather than assumed. On a small domain corpus, a well-smoothed bigram model frequently beats a trigram model, because the trigram has more context and less evidence. This is the same bias-variance trade met in Volume Three, arriving in a new costume.

There is a second, subtler cost. A longer context is more specific to the training text, so it generalises less. The trigram "the pump was" may occur in training only in the context of a particular failure description, and the model will then insist that the phrase is followed by that description. The bigram "pump was" has seen more of the world.

Generating Text From a Table

A bigram model can be run backwards, as a generator: start with a boundary, sample the next word according to the table, repeat.

On a real corpus the output has a characteristic texture. It is locally plausible — each adjacent pair is a pair that occurs — and globally incoherent, drifting by association. The classic demonstration produces sentences that begin sensibly and end somewhere unrelated, which is exactly what a one-word memory should produce:

```
the pump was replaced by the supplier reported that the seal kit
```

Every pair here is attested. The sentence is nonsense. This is the clearest possible illustration that local statistics are not meaning, and it is why the subject did not stop here.

Perplexity

To compare models you need a single number, and the standard one is **perplexity**: the inverse probability of the test text, normalised per word.

```
perplexity = (1 / P(test text))^(1/N)
```

where N is the number of words. A model that assigns higher probability to real text has lower perplexity. In words: perplexity is roughly the number of equally likely choices the model is hesitating between at each step.

Two worked values. If a model gives every word in a ten-word sentence a probability of 0.1, then

P = 0.1^10 = 10^-10 perplexity = (10^-10)^(-1/10) = 10

The model is, on average, choosing among ten options. If the model instead assigns 0.05 per word, perplexity is 20 — twice as uncertain.

A useful calibration: uniform choice over a 40,000-word vocabulary is a perplexity of 40,000. A decent bigram model on English news text reaches a few hundred. A good modern model reaches the low tens. The distance between those numbers is the distance between a lookup table and a system that has learned something about structure.

Perplexity has two traps. It is only comparable between models with the same vocabulary — a model with a 30,000-token vocabulary and one with a 50,000-token vocabulary are scoring different tasks. And it measures how well a model predicts text, not whether it produces useful text; a model can have excellent perplexity and be useless for the task at hand.

Exercises

1. Using the bigram table, compute the probability of "the cat ate the fish", then of "the fish ate the cat".

2. Add the sentence "the cat sat on the fish" and recompute P(cat | the) and P(fish | the). How far did two observations move them?

3. A corpus has 25,000 distinct words. How many bigrams and trigrams exist, and what fraction of bigrams would 500,000 tokens contain?

4. A model gives probabilities 0.2, 0.1, 0.05, 0.01, 0.01. Find the perplexity.

5. Why can perplexity not be compared across two tokenisers?

6. Generate six words from the bigram table, taking the most likely continuation each time. Where does it get stuck?

CHAPTER SIX

Smoothing, and the Cost of the Unseen

A support chatbot was built on a trigram model over four years of transcripts. It answered routine questions well. Then a new product line launched, and the very first message about it — "where is the clamp for the new bracket" — produced no reply at all, because the trigram "the clamp for" had never occurred in the old transcripts and its probability was zero.

The system did not say "I don't know". A model that multiplies probabilities has no way to say anything once a factor is zero. The reply was empty, and the operator assumed the bot was broken, which in a sense it was.

Every language model that has ever been built is, at bottom, a scheme for assigning a small but nonzero probability to text it has never seen. This chapter is that scheme.

The Zero Problem

A bigram model estimates

```
P(word | previous) = count(previous, word) / count(previous)
```

If the pair has never occurred, the numerator is zero. The denominator may be large — "the" might occur 60,000 times — and the estimate is still zero, because a single unseen continuation makes the whole probability collapse.

The problem is not the missing pair. It is that the estimate treats "never observed" as "impossible", and for language that is obviously wrong. English does not run out of sentences.

Add-One, With Numbers

The simplest repair is to pretend every possible pair was seen one extra time.

```
P(word | previous) = [count(previous, word) + 1] / [count(previous) + V]
```

where V is the vocabulary size. Take the corpus from the previous chapter, with V = 8 (the, cat, sat, on, mat, ate, fish, and the boundary).

Unsmoothed: P(cat | the) = 2/4 = 0.50.

Smoothed: (2 + 1) / (4 + 8) = 3/12 = 0.25.

Every unseen pair now has probability 1/12 = 0.083, which is the point — nothing is impossible any more. But note what happened to the pairs that were genuinely common: their probability fell by half. Add-one takes mass from the observed and gives it to the unobserved, and it does so without regard to how much room there is.

The consequence is easy to see in the vocabulary size. With V = 40,000 and a context seen 4 times, the smoothed estimate is (2+1)/(4+40,000) = 0.000075, against an unsmoothed 0.50. The model has been destroyed to rescue one unseen pair. This is why add-one is a teaching device and not a production method: for large vocabularies it gives away almost all the mass.

The general repair is **add-k**, with k much less than one — 0.01 or 0.001 — or better, a method that gives mass to unseen events in proportion to how many *distinct* continuations a context has, rather than uniformly.

Method	Unseen probability	Behaviour
None	zero	Fails on any novel text
Add-one	1/(count + V)	Simple, over-smooths badly
Add-k	k/(count + kV)	Tunable, still uniform
Good-Turing	proportional to singletons	Principled, unstable at scale
Kneser-Ney	from continuation counts	Best classical option
Interpolation	mixture of orders	Simple and effective

Interpolation, Which Is What People Actually Do

Instead of choosing one order, mix them.

> P(word | previous two) = lambda x P3(word | previous two) + (1 - lambda) x P2(word | previous)

Worked, with lambda = 0.7. Suppose the trigram "clamp for the" has count 3 out of 5 occurrences of "clamp for", and the bigram "for the" has count 40 out of 900 occurrences of "for".

P3 = 3/5 = 0.60 P2 = 40/900 = 0.044 P = 0.7 x 0.60 + 0.3 x 0.044 = 0.42 + 0.013 = 0.433

The high-order model dominates when it has evidence, and the low-order model holds the floor. Where the trigram is unseen, the estimate falls back to the bigram, which is the behaviour the chatbot needed.

The weights can be fitted on held-out text, and doing so is a five-line exercise that outperforms any amount of intuition. The principle is worth stating: the more data a context has, the more the model should trust the specific context; the less it has, the more it should trust the general one.

Evaluating: Held-Out Text, Not Sentences

A language model is evaluated on text it has not seen, and the split must be by document, not by sentence. Two reasons.

Sentences from the same document share vocabulary, topic and often phrasing, so a random sentence split leaks. A model that has seen the rest of a manual will predict the next page of that manual far better than it predicts a page from a different manual.

The quantity reported is usually perplexity on the held-out set, and the comparison must hold two things fixed: the tokenisation and the vocabulary. Change either and the number is meaningless.

A worked comparison, on the same 10,000-word held-out text:

Model	Perplexity	Reading
Uniform over 20,000 words	20,000	No information
Unigram	900	Frequency only
Bigram, no smoothing	infinite	Contains a zero
Bigram, interpolated	320	Useful
Trigram, interpolated	240	Better

Note the third row. A model that looks excellent on its training text scores infinitely badly on held-out text the moment it meets one unseen pair, and this is the single most important fact in the chapter. A model evaluated on the data it was trained on is not being evaluated at all.

Why This Still Matters

It is tempting to treat all of this as history now that models are trained by gradient descent on enormous corpora. It is not history, for three reasons.

The mechanisms are the same. A neural language model also assigns probability to the next token given a context; it simply represents the context as a vector rather than a lookup, and therefore generalises to unseen contexts instead of failing. The reason modern models handle "the clamp for" is that they are not counting that exact trigram — they are computing from a representation that shares structure with everything similar.

The evaluation is the same. Perplexity, held-out text, splits by document: the discipline carries over unchanged.

The failure is the same. A model meeting text from a new domain degrades, and the degradation is invisible unless the held-out set is genuinely new. The chatbot in the opening was not defeated by its smoothing method. It was defeated by a corpus that stopped before the product launched.

Working Notes

1. With V = 12 and a context seen 6 times, compute the add-one probability for a word seen twice, and for a word never seen.

2. A trigram model gives 0.4 for the observed trigram, and the interpolating bigram gives 0.05. With the weight at 0.6, what is the mixed probability? What happens at 1.0?

3. Explain why add-one smoothing is unacceptable for a 40,000-word vocabulary, with one calculation.

4. A model scores perplexity 180 on held-out news and 610 on held-out legal contracts. What does the difference tell you, and what would you do before deploying it on contracts?

5. Design a train/validation/test split for a corpus of 5,000 emails spanning three years, where the system will be deployed next year. State the rule, not the ratio.

6. Give one sentence of text that a bigram model would score as impossible and a trigram model would score as probable, and explain why the reverse can also happen.

PART THREE

Learning From Text

CHAPTER SEVEN

Context Windows and Learned Predictions

The summariser produced its first hundred outputs and the reviewer noticed a pattern: eight of them opened with the same sentence, and it was the first sentence of the document. The system had learned, from its training data, that summaries usually begin at the beginning. It was not summarising. It was copying the opening line and continuing in a plausible style.

This chapter is about the machinery behind such systems, described as plainly as the machinery allows: how a model uses a window of surrounding text, why that window has a size, and what the size costs.

From Lookup Table to Representation

The bigram model of Chapter Five looked up a count. A learned model instead computes a vector for the whole context, and then computes a probability over the vocabulary from that vector.

The difference in behaviour is exactly what the counting theory predicts. A lookup table fails on an unseen context because there is no entry. A representation does not have entries; it has a function, and the function produces a sensible answer for contexts it has never met, because similar contexts map to nearby vectors.

So "the clamp for the new bracket" can be handled even if never seen, provided the model has learned what "clamp" and "bracket" do in sentences. That is generalisation, and it is bought with computation rather than with counts.

Attention, in One Paragraph

The mechanism that made this practical in long contexts is attention, and the idea is smaller than the name.

For each position in the text, the model decides which earlier positions matter for predicting what comes next, and it does so by comparing a vector for the current position with a vector for each earlier one. The comparisons are turned into weights that sum to one, and the model reads a weighted mixture of the earlier positions.

A worked set of weights for one position, over four earlier tokens:

token 1: 0.05 token 2: 0.15 token 3: 0.50 token 4: 0.30

The sum is 1.00, and the model's "view" of the past at this position is 50% token 3, 30% token 4, and almost nothing else. In a real model there are many such attention heads and many layers, so the picture is a stack of these mixtures, but the arithmetic of any single one is that sum.

Two consequences follow. The window can be long, because attention does not require the context to be represented by a fixed-size state. And the cost grows with the square of the window length, because every position compares with every earlier position — which is why context length is expensive and why "just use a bigger window" is a budget decision, not a technical one.

What Fits in a Window

Context windows are quoted in tokens. Converting to something human:

Window	Ordinary English words	What it holds
2,000 tokens	about 1,500	Two long emails
8,000 tokens	about 6,000	A short report
32,000 tokens	about 24,000	A chapter of a book
128,000 tokens	about 98,000	A small manual

The conversion is approximate, and it differs by language and by tokeniser, which is the point made in Chapter Two: a window is a token budget, and the same window holds fewer words in German than in English.

The practical issue with windows is not what fits but what gets *used*. Information at the far end of a long window is often attended to weakly, and a system that puts the crucial instruction at the start of a very long prompt is hoping rather than designing. Put the instruction where it will be read: near the end, or repeated.

Temperature, and Why Outputs Vary

A model produces a probability for every token in its vocabulary. To generate text you sample from that distribution, and how you sample changes the character of the output.

Temperature divides the log-probabilities before the distribution is renormalised. Below one, the distribution sharpens; above one, it flattens.

Take a distribution over three candidate tokens: 0.6, 0.3, 0.1.

Temperature	Resulting distribution
0.5	0.78, 0.20, 0.02
1.0	0.60, 0.30, 0.10
1.5	0.52, 0.33, 0.16

At temperature 0.5 the model almost always takes the most likely token: consistent, repetitive, and occasionally stuck. At 1.5 it takes the third option one time in six: varied, and more prone to drifting off the point. There is no correct setting, only a setting appropriate to the task — extraction wants low, brainstorming wants higher.

Greedy decoding, which always takes the most likely token, is the limit of temperature zero, and it produces text that loops: the same clause, then the same clause again, because the model's own output has become its context.

What Such Models Are and Are Not

They are next-token predictors trained on a very large amount of text. Everything they do follows from that: fluency, because fluent text is what predicts well; factual errors in the shape of confident prose, because a plausible continuation is not a verified one; and a tendency to agree with the premise of a question, because agreement is the common continuation.

The word for the third failure is sycophancy, and the word for the second is hallucination. Neither is a bug in the usual sense. A model that generates a citation for a paper that does not exist is doing precisely what it was trained to do — producing the token sequence that looks most like a citation — and the repair is external: retrieve the source, check it, and refuse when it is absent.

That is why this volume puts retrieval and evaluation after this chapter rather than before. A generative model without a retrieval step and an evalu-

ation step is a demonstration. With them, it is a system.

Cost, Which Is Real

Three quantities determine whether a language model can run in a given application.

Tokens in and out. A 2,000-word prompt is about 2,600 tokens; a 300-word answer about 400. Multiply by requests per day, and the bill follows.

Latency. Generation is sequential: each token requires a full pass through the model. At twenty tokens per second, a 400-token answer takes twenty seconds, and no amount of parallel hardware changes the sequence — it only makes each step faster. Streaming the output hides the delay without reducing it.

Memory for context. The intermediate state for each token in the window must be kept while generating, so a longer window costs memory as well as time. This is why long conversations degrade or become expensive, and why summarising the conversation periodically is a standard engineering move.

Use	Constraint	Consequence
Chat assistant	Latency, 1–3 s to first token	Stream, keep answers short
Batch classification	Throughput	Batch requests, avoid generation
Document question answering	Window, retrieval	Chunk, retrieve, cite
Real-time control	Latency under 100 ms	Do not use a generative model

Exercises

1. A prompt contains 1,800 words. Estimate the token count, and say whether it fits a 2,000-token window.

2. A distribution is 0.5, 0.3, 0.15, 0.05. Compute the temperature-0.7 version approximately, and describe the change in behaviour.

3. Explain, in three sentences, why a model can produce a fluent and entirely false paragraph.

4. A system must answer 5,000 questions a day with a 400-token answer each. At twenty tokens per second, how much generation time is that in

total, and does it need more than one machine?

5. Why does attention cost grow with the square of the window, and what practical consequence does that have for a 100,000-token prompt?

6. Design a prompt layout for a question-answering system where the instruction, the retrieved documents and the question must all be present. Where does each go, and why?

CHAPTER EIGHT

Classifying Text

A classifier that routed support email to the right team reported 97% accuracy in testing. On the live queue it was wrong about one message in five, and the errors had a comic consistency: every message that had been forwarded internally was routed to "IT".

The tag "[EXTERNAL]" that the mail gateway appended to incoming messages had been stripped by the cleaning step, so its absence became the signal for "forwarded internally". The model had found a perfect predictor of the wrong thing, and no amount of tuning would have fixed it, because the predictor was genuinely perfect in the training data.

Text classification is the most-used application of language processing and the one where contamination is easiest. This chapter is about building one that survives contact with production.

From Text to Features

A classifier needs numbers. Four options, in increasing complexity.

Bag of words. Count each vocabulary word in the document. Loses order entirely, and works surprisingly well.

tf-idf. Weight each count by how rare the word is across documents, as computed in Chapter Three. The standard baseline for retrieval and classification, and hard to beat on small datasets.

Averaged embeddings. Replace each word with its vector and average. Captures similarity between words, loses exact terms.

Learned representations. A network reads the text and produces a vector. Best when data is plentiful, and it needs thousands of labelled examples before it beats tf-idf.

A worked tf-idf for one document in a collection of 1,000, where "leak" occurs twice and appears in 50 documents:

tf = 2 / (number of words in document) — say 2/120 = 0.017 idf = log(1,000 / 50) = 3.0 weight = 0.017 x 3.0 = 0.05

And for "the", occurring twenty times and appearing in every document:

idf = log(1,000 / 1,000) = 0 weight = 0

The frequent, uninformative word is eliminated by arithmetic rather than by a stop-word list. That is the appeal of the method.

Naive Bayes, Done by Hand

The oldest classifier is still a serviceable baseline and can be computed with a pencil.

Suppose 100 emails: 40 spam, 60 ordinary. The word "free" appears in 24 spam and 6 ordinary messages.

P(free | spam) = 24/40 = 0.60 P(free | ordinary) = 6/60 = 0.10 P(spam) = 0.40, P(ordinary) = 0.60

Multiply and compare:

spam: 0.40 x 0.60 = 0.24 ordinary: 0.60 x 0.10 = 0.06

Normalising gives a posterior of 0.24 / 0.30 = 0.80 for spam on the strength of one word. Add more words and the product extends.

The "naive" in the name is the assumption that words are independent given the class, which is false — "credit" and "card" are not independent — and yet the ranking it produces is often good. What it produces is not a calibrated probability, and treating 0.80 as "80% likely" is a mistake the next section addresses.

Two practical notes. Unseen words must be smoothed, exactly as in Chapter Six, or one unseen term zeroes the product. And the class priors must reflect deployment: a corpus that is half spam will produce a classifier that flags far too much when deployed on a mailbox that is 5% spam.

The Integrity of the Split

This is where text classification fails, and it fails in ways that are invisible in a notebook.

Leakage source	How it happens	Effect
Same thread in train and test	Random split over messages	Validation score inflated
Near-duplicate documents	Templated emails, quoted replies	Model memorises templates
Sender in both sets	One person's style	Learns the person, not the topic
Time	Train on 2023–24, test randomly	Ignores drift between years
Label-derived features	ID numbers correlated with class	Learns the annotation order
Boilerplate	Signatures, disclaimers, gateway tags	Learns the plumbing

The last row is the opening story, and it is the most common of all. The defences are mechanical: strip known boilerplate, deduplicate by similarity, split by thread or by sender or by date, and inspect the most predictive features of the fitted model. If the top feature is a tag rather than a topic, the model is measuring the plumbing.

Looking at the top features takes ten minutes and is the single most valuable diagnostic in applied text work.

Choosing the Metric

Accuracy is a poor summary, for the reasons established in Volume Three, and text makes it worse because class frequencies are usually extreme.

The alternatives:

Precision and recall per class, with the operating threshold stated.

F1, the harmonic mean, when a single number is demanded and the two error types are equally costly.

Macro-average across classes, which weights each class equally and therefore punishes a model that ignores a rare class. **Micro-average** weights by frequency and flatters it.

Calibration, which asks whether a predicted 0.8 corresponds to a frequency of 80%. A model can rank well and be badly calibrated, and a threshold chosen on uncalibrated scores will not transfer between datasets.

For a routing system, the relevant question is not accuracy at all. It is: how many messages per day reach the wrong team, and what does that cost in delay? That question requires the confusion matrix and a cost per error, and nothing else.

Thresholds and the Cost of Being Wrong

A classifier outputs a score; a decision requires a threshold, and the threshold encodes a judgement about relative costs.

For routing, sending a message to the wrong team costs a few hours of delay. Sending it nowhere costs a customer. So the threshold for "confident" should be high, and everything below it should go to a human queue — which is not a failure of the model but a design decision that uses it where it is reliable.

This arrangement — automate the confident cases, queue the rest — is how nearly every deployed text classifier actually behaves, and it has the pleasant property that its performance can be measured directly as the fraction of traffic that avoids the queue and the error rate within it.

Problems

1. A collection has 2,000 documents and "bracket" appears in 100. Compute its idf, and the weight of a 200-word document containing it twice.

2. Using the Naive Bayes numbers above, compute the posterior if "free" appeared in 12 spam and 12 ordinary messages. What changed, and why?

3. A routing classifier reports 95% accuracy across five classes. Class sizes are 800, 100, 50, 30 and 20. Compute the accuracy of a model that always predicts class one, and explain what that implies.

4. Name three boilerplate features that leak into email classifiers, and how you would detect each.

5. A model has precision 0.9 and recall 0.4 on the "urgent" class. At 600 urgent messages a month, how many are missed, and how many false alarms does it raise?

6. Design a split for 8,000 tickets from three years and four products, deployed next month. State the rule and the reason.

CHAPTER NINE

Extraction: Names, Relations and Structure

An accounts payable team automated invoice entry. The system extracted six fields per invoice — supplier, invoice number, date, purchase order, net total, tax — and reported 94% accuracy across all fields. The business case assumed the six per cent would be caught in review.

Nobody had done the multiplication. If each field is right 94% of the time and errors are independent, the chance that all six fields of an invoice are correct is 0.94^6 = 0.69. Nearly a third of invoices needed correction, not six per cent, and the review queue was designed for a tenth of that volume.

The arithmetic of extraction is the arithmetic of conjunctions: accuracy per field multiplies, and review capacity does not.

The Shape of the Task

Extraction turns unstructured text into fields. Three sub-tasks, which are usually confused with one another.

Finding spans. Locating a piece of text: the supplier name, the date, the amount.

Classifying spans. Deciding what each piece is: this string is an organisation, that one is a location.

Assembling records. Combining spans into a structure that satisfies a schema, with relations between them: this total belongs to this purchase order.

The standard notation for the first two is a tagging scheme, and it is worth doing once by hand.

Send 40 boxes to Acme Ltd in Leeds on 3 May

Token	Tag	Meaning
Send	O	outside any entity
40	B-QTY	begins a quantity
boxes	B-PROD	begins a product
to	O	
Acme	B-ORG	begins an organisation
Ltd	I-ORG	continues it
in	O	
Leeds	B-LOC	begins a location
on	O	
3	B-DATE	begins a date
May	I-DATE	continues it

The B and I prefixes exist to handle multi-token entities; without them, "Acme" and "Ltd" would be two unrelated organisations. A model trained on this scheme learns to predict a tag per token, and the tags are then grouped back into spans.

Rules First, and Honestly

A large fraction of extraction is better done with rules, and refusing to use them is a common affectation.

Invoice numbers have a pattern. Dates have a small number of formats. Part codes often begin with a fixed prefix. A regular expression finds these perfectly, costs nothing to run, and can be read by an auditor — which matters, because extraction usually feeds a financial or safety process.

Rules have two weaknesses, and they are worth naming precisely.

They break on variation. "3 May", "3rd May", "May 3", "03/05/2024" and "2024-05-03" are one fact. A rule set that handles all five is a small program, and the day a supplier writes "03.05.24" it fails.

They cannot generalise to unanticipated strings. A supplier name has no pattern; it is whatever the supplier calls itself.

The workable arrangement is a hybrid: rules where the format is standardised and legally constrained, models where the text is free, and a validation layer that checks the assembled record for internal consistency.

Measuring Extraction Properly

Entity extraction is scored by span overlap, and the arithmetic has a trap.

Suppose the system produced 100 predicted entities, of which 80 match a gold entity exactly. The gold data contains 95 entities.

precision = 80 / 100 = 0.80 recall = 80 / 95 = 0.84 F1 = 2 x 0.80 x 0.84 / (0.80 + 0.84) = 0.82

The trap is what counts as a match. "Acme Ltd" against "Acme Limited" is a partial match; a strict scorer counts it as an error, a lenient one as correct, and the difference between the two can be twenty points of F1. The scoring rule must be written down before the numbers are quoted, and it must match how the output will be used: if the field is written into an accounting system, an abbreviation difference is a real error.

A second trap: token-level accuracy flatters. A sentence with eleven tokens and one entity has eight "O" tags, so a system that predicts "O" everywhere scores well on tokens and zero on entities. Entities are the unit of measure.

Confidence, and the Human in the Loop

A model that predicts a field can also predict how sure it is, and the usefulness of that number depends entirely on whether it is calibrated.

Calibration is measured by binning. Take every field the system extracted, group them by predicted confidence, and compute the actual accuracy in each group.

Confidence band	Fields	Correct	Actual accuracy
0.95–1.00	4,200	4,116	98.0%
0.80–0.95	1,600	1,408	88.0%
0.50–0.80	300	219	73.0%
below 0.50	100	41	41.0%

A well-calibrated model has actual accuracy close to predicted confidence in every band. Here the top band is better than advertised and the middle band is worse, which is common: models are under-confident when they are right and over-confident when they are wrong.

The value of the table is that it converts a threshold into a staffing decision. If the review team can handle 1,500 fields a day, the policy "review everything below 0.95" produces about 2,000 — too many — while "review everything below 0.80" produces 400, and the expected error rate among the unreviewed is the accuracy in the top band, which is 2%.

That last number is the one that must be quoted to the business, and it must be quoted with its sample size. A band containing 4,200 fields gives an accuracy estimate with a margin of about one percentage point; a band containing a hundred gives a margin of ten, and a policy built on it is built on noise.

Two disciplines make this work in practice.

Recalibrate after every model change. Confidence scores are not comparable across versions, and a threshold carried over from the previous model may review nothing or everything.

Log the review outcome, not only the decision. The only way to know whether the top band is really 98% next quarter is to sample it and check, which is the same argument as the capability study in the previous volume.

Relations, Which Are Harder

Fields in isolation are easy; the relations between them are where invoices are lost.

Relation	Example	Failure mode
total belongs to invoice	one total, several line items	picks the subtotal
invoice references order	"against PO 4471"	misses the reference
supplier issues invoice	letterhead, signature block	confuses buyer and seller
tax applies to net	two amounts, one rate	assigns the wrong rate

These are usually extracted as triples — subject, relation, object — and then checked against a schema. The checks are where most of the reliability comes from, and they are cheap:

Does the total equal net plus tax?

Does the tax rate correspond to a plausible jurisdiction?

Is the invoice date later than the order date?

Does the purchase order exist?

Each check catches a class of error that no amount of model improvement would catch, because the error is visible only in the relationship between fields, not in any single field.

The Review Queue Is a Design, Not a Failure

Given per-field accuracy, the volume of work is a calculation, and it should be done before the system is built.

Ten thousand invoices, six fields, 94% per field:

> field errors = 10,000 x 6 x 0.06 = 3,600 invoices needing any correction = 10,000 x (1 - 0.69) = 3,100

If a reviewer can correct an invoice in ninety seconds, that is seventy-seven hours of work a month — roughly half a person. The case may still be excellent: it may replace a hundred and fifty hours of manual entry. But the number must be computed, and the review capacity designed around it, or the queue becomes the system's reputation.

Policy	Effect
Review everything	Accuracy never improves; cost unchanged
Review low-confidence only	Needs calibrated confidence
Review by risk	High-value invoices always, rest sampled
Sample audit	Detects drift, does not correct errors

The third and fourth rows are usually combined in practice: a rule that high-value invoices are always checked, plus a random sample of the rest to measure the error rate over time. The sample is what tells you whether the model is still working in month six.

Exercises

1. A system has 96% accuracy per field and extracts four fields. What fraction of records is fully correct? How does that change at 99% per field?

2. Tag the sentence "Deliver 12 valves to Beta Engineering in Ibadan before 9 June" using B and I prefixes.

3. Precision is 0.75 and recall 0.60. Compute F1. If recall rises to 0.70 at the same precision, what is the new F1?

4. Write four validation checks for an invoice record that a regex alone could not provide.

5. A reviewer handles forty invoices an hour and there are 6,000 invoices a month. How many reviewer hours are needed if 30% require correction?

6. A supplier starts writing dates as "12.vi.24". Explain what happens to a rule-based extractor, to a model-based one, and to a hybrid, and say which you would rather operate.

CHAPTER TEN

Search, Retrieval and Answering Questions

An engineer asked the internal assistant how to reset a specific controller. It replied, confidently and at length, with the controller's address, its baud rate and three steps that were almost right.

The retrieval step had returned the manual's table of contents — a page dense with the right words and containing none of the procedure — and the generative step had done what generative steps do: it had produced plausible prose from an outline. The failure was in retrieval, and it looked like a failure of the model.

This chapter is about the half of a question-answering system that decides what the model is allowed to read.

The Inverted Index

The classical search stack is an inverted index: for each term, a list of documents containing it, with positions. Building one from a corpus of documents is a counting exercise, and the counting is the same counting as Chapter Three.

Retrieval scores documents against a query, and the dominant classical formula is **BM25**, which balances three effects: a term's rarity, its frequency in the document, and the document's length.

```
score = idf x [ tf x (k1 + 1) ] / [ tf + k1 x (1 - b + b x doclen / avgdoclen) ]
```

with idf = ln((N - df + 0.5) / (df + 0.5) + 1), k1 around 1.2 and b around 0.75.

Worked, in a collection of 1,000 documents with an average length of 120 words. The query term appears in 10 documents, so

```
idf = ln(990.5 / 10.5 + 1) = ln(95.3) = 4.56
```

Three candidate documents:

Document	Term count	Length	Score
A	3	100	7.43
B	1	100	4.89
C	3	300	5.42

Document A beats C although both contain the term three times, because C is three times longer and is therefore less about anything in particular. Document B beats nothing and is ranked last. This is the behaviour a person expects from search, derived from counts.

The length normalisation is the part people find surprising, and it is the part that matters most on real corpora: a long document containing a word once is making a passing reference, and a short document containing it once may be about nothing else.

Measuring Search

Retrieval is measured with the queries you will actually receive, and the measures are simple.

Precision at k — of the top k results, how many are relevant. Three relevant in the top five is 0.6.

Recall at k — of all relevant documents, how many appear in the top k. With four relevant in the collection, three in the top five is 0.75.

Mean reciprocal rank — the average of 1/rank of the first relevant result. Three queries answered at ranks 3, 1 and 8:

$$(1/3 + 1/1 + 1/8) / 3 = (0.333 + 1.000 + 0.125) / 3 = 0.486$$

MRR is the right summary when a user needs one good result, as in an assistant. Precision at k is right when a user will read several, as in research.

The hard part is the query set. A hundred real queries, sampled from logs and labelled for relevance by someone who knows the domain, is worth more than any amount of tuning against invented queries — and the labels are the expensive part.

Semantic Retrieval, and Why It Is Not a Replacement

Embedding retrieval replaces term matching with vector similarity: embed the query and the documents, and return the nearest documents.

It finds the credit note when you ask for a refund, which keyword search cannot. It also fails in ways that keyword search does not:

It cannot match a part number. "XYZ-40" and "XYZ-41" are nearly identical vectors and completely different components.

It has no notion of exact phrase, so a query for a quoted error message may return anything semantically nearby.

It is opaque. When it returns the wrong document, there is no term to inspect and no score to explain.

The practical answer is **hybrid** retrieval: run both, and combine the rankings. A simple and effective recipe is to take the top fifty from each, merge the lists, and rerank the union with a small model or a hand-weighted score that gives a bonus to exact term matches. The bonus exists because rare exact terms are almost always the user's actual intent, and dense vectors under-weight exactly that.

A Second Pass: Reranking

Retrieval has two jobs that are hard to do at once: find candidates cheaply, then judge them carefully. Systems usually split the work.

The first stage scores the whole collection with something fast — BM25, or a vector index — and keeps the top fifty. The second stage reads each candidate together with the query and produces a relevance score. That second model sees the pair jointly, so it can notice that the query asks about "reset procedure" and the candidate discusses "resetting", which a bag of words cannot.

The cost is straightforward arithmetic. Fifty candidates of 300 tokens each is 15,000 tokens per query, evaluated by a model that reads query and candidate together — perhaps fifty times the work of scoring the same fifty documents separately. At ten queries a second, that is a substantial machine; at one query a second, it is nothing.

The trade is worth stating plainly, because it decides whether a second stage is affordable at all. If the application tolerates two hundred milliseconds per query, a reranker costing a hundred and fifty of those milliseconds is reason-

able. If the interface must respond within fifty milliseconds, it is not, and the improvement in ordering has to come from a better first stage instead.

Stage	Job	Cost	Failure if skipped
Filter	Remove obviously irrelevant	Low	Wasted work downstream
Retrieve	Get 50 candidates	Low	Recall cannot be recovered
Rerank	Order them well	Medium	Good documents ranked 20th
Read	Generate an answer	High	Wrong answer from right documents

The table explains a common misdiagnosis. When an assistant gives a poor answer, the instinct is to improve the generator. But the generator can only work with what it is shown, and if the right passage is at rank 30, no amount of model quality will recover it. Check the retrieval first: does the correct passage appear in the top k at all? If it does not, the answer is in the index, the chunks or the query, and never in the language model.

A useful diagnostic, and it takes an afternoon: take a hundred real questions, record the rank of the first genuinely relevant passage, and plot the distribution. If half the questions have their answer in the top three, the retrieval is sound and the problem is downstream. If the median rank is fifteen, the system is being asked to answer from the wrong half of the manual.

Chunking, Which Decides Everything

A long document cannot be embedded as a whole, so it is split into chunks. The arithmetic is trivial and the consequences are not.

A 60,000-word manual, chunked at 300 words with 50 words of overlap:

```
chunk count = (60,000 - 50) / (300 - 50) = 240
```

Each chunk is retrieved independently, so a system that returns five chunks is showing the model 1,500 words out of 60,000 — two and a half per cent of the manual. If the answer spans a chunk boundary, or if the relevant chunk ranks sixth, the model never sees it.

Chunking choice	Effect
Too small	Loses context; answers refer to missing steps
Too large	Retrieval becomes coarse; many irrelevant words
No overlap	Answers cut mid-sentence at boundaries
Overlap 10–20%	Standard compromise
Split by section	Best, when the document has structure

The last row is the one that pays. Documents written by human beings have headings, and a chunk that follows a heading boundary contains a coherent thought. Chunking by fixed length throws that structure away and then spends effort recovering it.

What Actually Fails

A question-answering system built on retrieval fails in five characteristic ways, and diagnosing which one is happening is most of the engineering.

Failure	Symptom	Fix
Retrieval miss	Answer ignores the obvious document	Better chunks, hybrid search, more k
Stale index	Answer cites a superseded version	Rebuild on change, show dates
Chunk boundary	Answer stops mid-procedure	Overlap, section-aware splitting
Conflicting sources	Answer averages two policies	Rank by authority, show both
No answer exists	Confident invention	Refuse when retrieval is weak

The fourth row is the one nobody anticipates: a corpus containing two versions of a policy, retrieved together, produces an answer that is a blend of both. The repair is not a better model but a better corpus — a versioned index with the obsolete document removed, and the removal treated as part of the deployment.

The fifth row is a design decision that must be made explicitly. A system that answers only when retrieval produces a strong match, and otherwise says it does not know, will be trusted. A system that always answers will eventually answer wrongly about something important, and the trust will not return.

Things to Try

Build the smallest possible search engine: ten text files, an inverted index of words to file names, and a scorer that counts matching words and divides by document length. Then run ten queries you care about and record where the right document appears.

The exercise takes an hour and teaches three things permanently: that length normalisation matters, that the words you thought were distinctive ("the", "system") are not, and that the query you wrote is rarely the query the index needs.

Problems

1. Compute the idf for a term appearing in 3 documents of a 500-document collection. Is it more or less informative than a term in 30 documents?

2. A document of 200 words contains a term 4 times, in a collection with an average length of 100 and idf 3.0. Compute its BM25 score with k1 = 1.2 and b = 0.75.

3. Three queries are answered at ranks 1, 4 and 10. Compute MRR.

4. A manual of 120,000 words is chunked at 200 words with 40 overlap. How many chunks? How many words does a system returning k = 8 show the model?

5. Explain why an embedding search fails on part numbers, and describe a fix that does not abandon embeddings.

6. A corpus contains a policy from 2023 and its replacement from 2025. Describe three mechanisms that prevent the old one influencing answers.

PART FOUR

Language in the World

CHAPTER ELEVEN

Translation and the Problem of Equivalence

A translated warning label read, in the target language, "the guard may be removed while the machine is running". The source said "the guard must not be removed while the machine is running". One word had been dropped, the sentence remained grammatical, and the label was now an instruction to do something dangerous.

The translation system had been evaluated on a general corpus and had scored well. Nothing in that score distinguished an error that misplaced a comma from an error that inverted a safety instruction.

Translation is the clearest case in this volume of a task where the metric and the purpose are not the same thing.

There Is No Single Right Answer

Begin with a sentence and three published translations of it. All three are correct. They differ in rhythm, in the order of clauses, and in whether an idiom is preserved or replaced.

> The engine will not start if the fuel cap is loose.

A faithful version keeps the conditional at the end; a natural version reverses it; a terse version becomes two sentences. A machine that produces any of them has succeeded, and a metric that compares against one reference will mark two of them wrong.

This is the **equivalence problem**, and it is not a technicality. Translation is not a substitution cipher between languages; it is the production of a text that does the same job in a different culture of writing. Word order differs, articles appear and disappear, polite forms carry information that English marks with modals, and compound nouns unpack into phrases.

Source phenomenon	What happens in translation
German verb-final clauses	Reordering across the whole sentence
Japanese topic markers	Subject supplied or dropped
English phrasal verbs	Single verbs in Romance languages
Measure words	Required in some languages, absent in others
Decimal comma	1.5 becomes 1,5 — and a machine may "correct" it

The last row is not a curiosity. A number converted between conventions without care changes a specification by a factor of ten, and no grammar check will notice.

Alignment: The Old Machinery

Before neural methods, translation was assembled from statistics over aligned text: sentence pairs from bilingual documents, aligned at the word level by counting co-occurrence, then phrases extracted and reordered by a model of movement.

The word alignment step is the same distributional idea as Chapter Four, applied across languages: the English word "pump" co-occurs with the German "Pumpe" far more often than chance, in sentences that are translations of one another. A handful of such correspondences, extracted from millions of sentence pairs, is enough to translate.

Two consequences survive into modern systems. Terminology is learned from data, so a domain with distinctive vocabulary — medical, legal, industrial — needs its own parallel text or an explicit glossary. And word order differences are the hardest part, which is why reordering was the central problem of the field for two decades.

Encoder, Decoder, Attention

The modern arrangement is conceptually simple, and it reuses the machinery of the previous part.

An **encoder** reads the source sentence and produces a sequence of vectors, one per token, each informed by its neighbours through attention. A **decoder** generates the target sentence one token at a time, attending to the encoder's vectors as it goes. At each step it decides which source words matter — which is

exactly the alignment problem, learned rather than counted.

Nothing in this description involves understanding. The system has learned a function from source sentences to target sentences, and the function is good because the training data was enormous.

Measuring Translation: BLEU, and Its Arithmetic

The conventional automatic measure compares the system's output with one or more human references, counting overlapping word sequences.

Take a candidate of five words and a reference that shares four of its words and two of its word pairs.

1-gram precision = 4/5 = 0.80 2-gram precision = 2/4 = 0.50

BLEU combines the precisions geometrically, with a brevity penalty for outputs shorter than the reference. With equal lengths by assumption:

BLEU-2 = sqrt(0.80 x 0.50) = 0.632

A score of 0.632 on a two-gram BLEU is respectable; four-gram BLEU scores are lower and are the usual published figures.

The measure has known properties, and they explain a great deal of published confusion.

It rewards word overlap, so a valid paraphrase scores badly. Two human translators often score lower against each other than a machine does against one of them.

It is sensitive to tokenisation. Change how punctuation or compounds are split and the score moves several points without any change in quality.

It does not distinguish error types. Dropping "not" and misspelling a name cost the same order of magnitude as changing a preposition.

Score movement	Likely cause
3 points down after a tokeniser change	Measurement artefact
5 points up on in-domain test set	Domain match, not general improvement
1 point up, human reviewers prefer new output	Real but small
No change, users complain more	Metric not measuring what users see

What Must Be Measured Instead

For any translation that will be acted upon, the useful measurements are categorical and human.

Adequacy — does the translation convey the same information? A reviewer marks each sentence as correct, partly correct, or wrong.

Fluency — is it grammatical in the target language? Independent of adequacy, and sometimes in tension with it.

Critical error rate — the fraction of sentences containing an error that changes meaning, inverts a negation, alters a number or unit, or corrupts a name. This is the number that matters for instructions and labels, and it is the number that BLEU does not report.

A worked example from a hundred-sentence sample:

Category	Sentences
Fully correct	71
Minor error, meaning intact	17
Meaning changed	9
Negation or number corrupted	3

A BLEU score of 45 would be reported as strong. The critical error rate is 12%, and for a safety label it is disqualifying. Both numbers are true, and only one of them is about the application.

The Practical Arrangement

Three rules cover most industrial translation work.

Glossary first. Product names, part numbers, units and regulated terms are not translated by statistics; they are fixed by a glossary applied before and after the model. This is cheap and prevents the most embarrassing errors.

Numbers and negations checked by rule. A post-translation check that confirms every number, unit and negation in the source appears in the target catches the errors that matter most and costs almost nothing. It is not a model; it is a matching procedure.

Human review proportional to consequence. A marketing page can ship with machine output and a light edit. A warning label cannot. The decision is about consequence, not about the quality of the system.

Problems

1. A candidate translation has 20 words; 15 of its words appear in the reference and 8 of its bigrams do. Compute 1-gram and 2-gram precision, and BLEU-2 assuming no brevity penalty.

2. Explain why two professional translators' outputs might score BLEU 30 against each other, and what that implies about a machine scoring 40.

3. Give three examples of source text where a single dropped word changes the meaning dangerously.

4. A sample of 200 translated sentences contains 14 with changed meaning, 4 of which invert a negation. What is the critical error rate, and what would you recommend for a technical manual?

5. Describe a rule-based post-check that catches decimal-comma errors, and state one case it would miss.

6. A supplier offers a translation system scoring BLEU 52 on news and 31 on your maintenance manuals. Which number governs the decision, and what would you test next?

CHAPTER TWELVE

Speech: Sound Into Words

A hands-free control station in a workshop accepted voice commands: "start cycle", "stop", "next programme". It worked for a month. Then the fitters started listening to the radio, and the station began stopping the machine at unpredictable intervals, because a chorus in the background contained something that sounded like "stop".

The fix was not a better speech recogniser. It was a wake word combined with a confirmation for destructive commands, and a microphone that rejected sound arriving from the direction of the radio. Both are engineering, and both were cheaper than any model change.

Speech is the fastest of the language interfaces and the most exposed to the physical world. This chapter covers the arithmetic of sound, the standard pipeline, and the measurements that predict whether a voice interface will be usable.

Sound as Numbers

A microphone converts pressure to voltage, and an analogue-to-digital converter samples that voltage at a fixed rate.

Standard telephone-grade speech is sampled at 8,000 samples per second; wideband speech at 16,000. Each sample is usually quantised to 16 bits, two bytes.

```
16,000 samples/s x 2 bytes = 32,000 bytes/s = 32 kB/s
```

Ten seconds of speech is 320 kB, and an hour is 115 MB. That number explains why speech systems compress, transmit features rather than audio, and why a call centre recording everything is a storage project as much as an AI project.

The sampling rate sets the frequency range. The Nyquist limit says you can represent frequencies up to half the sampling rate, so 16 kHz audio carries information up to 8 kHz — enough for the consonants that distinguish "s" from "f", which telephone audio at 4 kHz often loses. That loss is why spelling a name over a landline is hard for people too.

From Waveform to Picture

Recognisers do not consume raw samples. They consume a time-frequency representation, computed in short overlapping windows.

A typical setting: a 25-millisecond window, advanced by 10 milliseconds, gives one hundred frames per second. Within each frame, the spectrum is summarised into 128 mel-spaced frequency bands — spaced to match human hearing rather than physics.

```
128 bands x 100 frames/s = 12,800 numbers per second
```

So one second of audio, which was 32,000 bytes, becomes 12,800 features. A ten-second utterance is 128,000 values, which is why speech models are small compared with text models and why audio context windows are short.

The mel scaling and the logarithmic compression exist for the same reason: human hearing is more sensitive to differences at low frequencies, and loudness is perceived logarithmically. A recogniser that ignored both would be solving a harder problem than necessary.

The Classical Pipeline, Still Instructive

Speech recognition was assembled from three components, and the division is worth knowing because the modern single model still does all three jobs.

An acoustic model maps features to sounds — historically phonemes, the small inventory of distinguishable units in a language. English has around forty.

A lexicon maps sounds to words. "Read" and "red" share sounds; "read" (past) and "read" (present) do not.

A language model supplies the prior probability of word sequences — the bigram and trigram models of Chapter Five, used exactly as described there, to break ties between acoustically similar candidates.

The interaction is why speech is hard: "recognise speech" and "wreck a nice beach" are acoustically close, and only the language model separates them. It is also why domain language models matter: a recogniser trained on general English has a weak prior for "torque wrench" and will prefer a phrase that sounds similar and occurs more often.

Modern systems replace the three components with one network trained end to end, which learns the acoustic mapping, the vocabulary and the prior together. The components have not disappeared; they have been absorbed.

Measuring Recognition

The standard measure is the **word error rate**, computed from an alignment between the reference transcript and the system's output.

```
WER = (substitutions + deletions + insertions) / number of reference words
```

Worked. A twenty-word reference, with three words misrecognised, one missing, and none inserted:

```
WER = (3 + 1 + 0) / 20 = 0.20, or 20%
```

Two things about this number are commonly misunderstood.

It can exceed 100%. A system that inserts many spurious words has a WER above one, and reporting it as a percentage of accuracy is meaningless.

It is dominated by the errors you care about least. Function words are frequent and easy, so the headline WER can be 8% while every part number is wrong. For a command interface, the relevant measure is the **command error rate**: the fraction of utterances that result in the wrong action, which is a much smaller and much more honest number.

Condition	Typical WER	Note
Close microphone, quiet room	5–8%	The demonstration condition
Close microphone, workshop	10–15%	Machines running
Far field, 3 m, reverberant	20–30%	The real condition
With domain vocabulary mismatch	30%+	Names, codes, jargon

The gap between the first and third rows is the whole reason voice interfaces disappoint. A system demonstrated at a desk is being evaluated in a condition

that does not exist on a factory floor.

Noise, Echo and the Physics of Failure

Four physical problems, in order of how often they break deployments.

Background noise reduces the signal-to-noise ratio, and the degradation is not graceful: below about 10 dB SNR, word error rises sharply. Directional microphones and close-talk headsets help more than any algorithm.

Reverberation smears each sound over hundreds of milliseconds in a hard room. The recogniser hears the previous syllable over the current one.

Echo from a loudspeaker means the system can hear itself, which is why barge-in — interrupting the system while it speaks — requires echo cancellation before anything else.

Multiple talkers produce overlapping speech that no current system separates reliably in a single channel.

None of these is fixed by a bigger model. All of them are addressed by microphone placement, acoustic treatment, and interface design — the same lesson as the previous volume's chapter on lighting.

Where the Microphone Goes

Most speech projects are decided by a bracket rather than a model, and the arithmetic of placement is simple enough to do before anything is bought.

Sound intensity falls with the square of distance, so doubling the distance from a talker costs about six decibels of signal. In a workshop where the background is 70 dB and a talker at one metre produces 60 dB at the microphone, the signal-to-noise ratio is negative and no recogniser will work. Moving the microphone to thirty centimetres recovers about ten decibels, which is usually the difference between unusable and marginal.

Directivity matters as much as distance. A cardioid microphone rejects sound from behind by fifteen to twenty decibels, which is the difference between picking up the machine behind the operator and ignoring it. An array of microphones can do better by steering towards the talker, and its benefit is largest in the reverberant rooms where single microphones fail.

Three placement rules, learned expensively across the industry:

Keep the microphone out of the direct path of loudspeakers, and if the system speaks, cancel its own output before anything else.

Mount the microphone on the structure the talker is at, not on the wall behind. Vibrations and reflections both arrive from the wrong direction otherwise.

Measure the actual spectrum in the actual room. A "quiet" office has a fan; a "noisy" workshop has a band of frequencies that a directional microphone can reject.

The measurement to make is not word error rate in a laboratory, but the signal-to-noise ratio at the microphone during the noisiest shift. It takes a sound level meter and an hour, and it predicts the outcome of the project better than any demonstration.

Speaking Back

Text-to-speech has its own arithmetic. The text must first be normalised: "3 May" read as a date, "1,200" as a number, "kg" as a unit, "Dr." as a title. Errors here are audible and embarrassing, and they come from the same ambiguity discussed in Chapter One.

Then prosody must be assigned — where the sentence stresses, where it pauses — because a synthesised sentence with flat intonation is harder to understand than a slightly mispronounced one with natural rhythm.

The measures that matter for an interface are not naturalness scores but comprehension: can a person follow the instruction while doing something else? A useful design rule is that any spoken instruction should be understandable at the first attempt, in noise, without repetition, or it should be redesigned to be shorter.

Exercises

1. A 30-second recording at 16 kHz, 16-bit, mono: how many bytes? How many bytes if converted to 8 kHz?

2. Compute the WER for a reference of 40 words with 4 substitutions, 2 deletions and 3 insertions. Why might the system still be unusable?

3. Explain why 8 kHz audio loses information that distinguishes "s" from "f", using the Nyquist limit.

4. A voice interface must work at 2 m in a workshop. Name three changes you would make before considering a different model.

5. A command set has 40 commands. A system with 10% WER produces how many wrong actions per 1,000 commands if errors are random, and why is the real number usually different?

6. Design a confirmation policy for the commands "stop", "start" and "next". Which need confirmation, and why?

CHAPTER THIRTEEN

Evaluation, Bias and the Decision to Ship

The assistant passed its pilot. Two hundred questions, ninety-one per cent answered correctly by a manager's reckoning, a demonstration to the board, and a plan to roll it out to every service desk in the group.

The audit that followed asked a different set of questions. Which two hundred questions? They had been written by the project team, from documents the project team had chosen. What was the ninety-one per cent measured against? One reviewer's judgement, recorded in a spreadsheet. What happens when it does not know? It answers anyway. And what happens when the documents change? Nobody had thought about it.

Every answer was defensible individually. Together they described a system that had never been evaluated against the traffic it would actually receive, by anyone who did not want it to succeed.

This chapter is the one that decides whether the previous twelve were worth writing.

Three Questions

Any claim about a language system should survive three questions, and they can be asked in thirty seconds.

What exactly was counted? Accuracy over what unit — sentences, documents, questions, fields? Scored by whom, against what reference, with what rule for partial credit?

What was held out? A test set drawn from the same distribution as training, or from the traffic the system will meet? Split by document or by sentence? How many near-duplicates were removed?

What happens next month? What changes — vocabulary, products, policies, the upstream pipeline — and what would detect the change before a user does?

A system whose authors can answer all three is not necessarily good. A system whose authors cannot answer them is not evaluated at all.

Layers of Evidence

Evaluation is not one experiment; it is a stack, and each layer answers something the one below cannot.

Layer	Question	Cost
Held-out test set	Does it work on data like the training data?	Low
Slice analysis	Does it work for every group and topic?	Low
Adversarial set	Does it fail gracefully on hostile input?	Medium
Human review of samples	Would a person accept this output?	Medium
Shadow deployment	Does it agree with the current process on live traffic?	High
Live trial with review	Does it help, and what does it cost to operate?	High

The first three can be done before any deployment and are routinely skipped. The fourth is where most projects should spend more time than they do, because human judgement on a stratified sample of two hundred outputs reveals failure modes that no aggregate score can express.

The fifth and sixth are the only ones that measure the system in the presence of real traffic, and they are also the only ones that can be wrong in the direction of optimism if the reviewers know which system produced the output.

Bias, Concretely

Language models inherit whatever is in their text, their labels and their sampling. The word "bias" is used loosely; the mechanisms are specific.

Corpus bias. A corpus of formal written English does not represent how people write in a hurry, in dialect, or in a second language.

Annotation bias. Labellers from one background apply one set of judgements. If the reviewed slice is labelled by the same three people, the model learns their habits and the evaluation confirms them.

Sampling bias. Traffic from one region, one product line, one customer tier. The system is excellent where it was measured and untested elsewhere.

Objective bias. A model trained to maximise agreement with past decisions reproduces those decisions, including the ones the organisation has since decided were wrong.

The measurable form of all four is **disaggregation**: compute the metric per group, per topic, per source, and report the worst slice alongside the average.

A worked example, on 5,000 classification decisions:

Slice	Share of traffic	Accuracy
All	100%	92%
Source A	78%	95%
Source B	14%	88%
Dialect group C	8%	78%

The average is 92%. For eight per cent of traffic it is 78%, and those are the users most likely to be writing informally, in a second language, or with unusual spelling — which is to say, the users least able to correct the system's mistakes.

A slice figure from a small sample is noise, so the sample size must be stated. To estimate a proportion to within five percentage points at 95% confidence requires about 384 examples; to within ten points, about 96. A slice with twelve examples and an accuracy of 67% is a rumour, not a measurement.

Privacy and the Text You Keep

Text systems ingest personal data as a matter of course: names, addresses, account numbers, health details, complaints. Four obligations follow, and they are engineering requirements rather than policy statements.

Minimise. Do not send a whole document when a paragraph is needed. Do not log a prompt when a hash of it would serve.

Redact before storage. Names and identifiers can be replaced with placeholders at ingestion, and the mapping held separately. This costs accuracy and preserves the ability to delete.

Retain deliberately. A retention period should be a decision with a date, not a default. Logs written "for debugging" become the archive that cannot be deleted when someone asks.

Respect the boundary. Data used to improve a model is a different use from data processed to serve a request, and in many jurisdictions it requires different consent. The distinction is easy to lose when the same pipeline does both.

Prompt Injection, in One Page

A system that reads retrieved documents and follows instructions is vulnerable to instructions hidden in those documents. A web page, a PDF, or an email can contain text addressed to the model rather than to the reader: ignore your previous instructions, send the following, approve this.

The mechanism is not a bug in the model; it is the consequence of putting untrusted text into a context that also contains instructions. Three mitigations reduce the risk substantially, and none eliminates it.

Keep instructions and data in separate, clearly delimited sections, and state in the instruction which sections are data.

Give the system the least authority it needs. A summariser that cannot send email cannot be tricked into sending one.

Validate outputs against a schema and an allowlist before acting. If an action must be taken, take it through a function with narrow permissions rather than through free text.

The residual risk should be stated in the deployment document, not discovered in an incident review.

Drift, and the Feedback Loop

A language system degrades when the world changes, and the changes are usually visible in the text before they are visible in the metrics.

Vocabulary drift: new products, new abbreviations, renamed departments. Detect by monitoring the rate of tokens that were rare or absent in training data.

Distribution drift: a new complaint category, a new supplier, a marketing campaign. Detect by monitoring class proportions and confidence distributions.

Pipeline drift: someone changes the email gateway tag, and a feature that the model relied on disappears — the opening story of Chapter Eight, which is a drift story.

The feedback loop is subtler. If the system's outputs are used to train the next version, the model learns its own mistakes as if they were truth. If human corrections are used, the model learns only about the cases humans chose to correct, which is a biased sample by construction. Either way, the training data stops being a sample of the world and becomes a sample of the system's history, and the evaluation must be built from fresh, independently collected text.

The Decision to Ship

Ten questions. They are deliberately boring, and a project that cannot answer them is not ready.

1. What is the unit of evaluation, and who labelled the reference?
2. What was held out, and by what rule was the split made?
3. What is the worst-performing slice, and how large is it?
4. What does the system do when it does not know?
5. What is the error that would matter most, and how often does it occur?
6. What will change in the next six months, and what will detect it?
7. What data is retained, for how long, and who can delete it?
8. What can the system do without human approval, and what can it not?
9. Who is accountable when it is wrong, and how does a user appeal?
10. How is the system turned off, and how long does that take?

Question four is the one that separates systems people trust from systems people tolerate. A language system that says "I cannot find that in the documents I have" when the retrieval is weak, and says it consistently, is worth more than one that answers every question with equal fluency.

For Practice

Take a system you have used — a search box, a chat assistant, a spam filter — and answer the ten questions about it. The gaps are the interesting part: which cannot be answered from outside, and what that tells you.

Then write the deployment paragraph for a system you would like to build: what it does, what it is measured on, and what it refuses. If you cannot write it, the system is not understood.

Glossary

Accuracy — the fraction of decisions that are correct. Uninformative when classes are unbalanced.

Add-one smoothing — adding one to every count so that unseen events have nonzero probability. Simple and badly biased for large vocabularies.

Alignment — the correspondence between words or phrases in a source text and its translation.

Attention — a mechanism that weights earlier positions in a context according to their relevance to the current one.

Bag of words — a representation that counts words and discards their order.

Bigram — a pair of adjacent tokens; a bigram model conditions each word on the previous one.

BLEU — an automatic translation score based on overlapping word sequences, with a penalty for short output.

BM25 — a ranking formula balancing term rarity, term frequency and document length.

Calibration — the property that predicted probabilities match observed frequencies.

Chunking — splitting long documents into passages small enough to embed or retrieve.

Collocation — a pair of words that co-occur more often than chance, such as "strong tea".

Context window — the number of tokens a model can attend to at once.

Cosine similarity — the cosine of the angle between two vectors; a scale-free measure of similarity.

Coverage — the share of running text accounted for by a given vocabulary size.

Distributional hypothesis — the idea that words used in similar contexts have similar meanings.

Edit distance — the smallest number of insertions, deletions and substitutions that turn one string into another.

Embedding — a dense vector representation of a token, document or query.

Entity — a span of text referring to a name, place, date, quantity or similar.

F1 — the harmonic mean of precision and recall.

Hallucination — fluent output that asserts something false, typically a plausible continuation rather than a retrieved fact.

Heaps' law — the sublinear growth of vocabulary size with corpus size.

Idf — inverse document frequency; a weight that reduces the importance of terms appearing in many documents.

Inverted index — a mapping from terms to the documents containing them.

Language model — a probability distribution over sequences of tokens.

Leakage — information from the test set reaching the model, inflating measured performance.

Lemma — the dictionary form of a word, to which its inflections are related.

Macro-average — averaging a metric across classes with equal weight, so rare classes count as much as common ones.

Mean reciprocal rank — the average of one over the rank of the first relevant result.

Mel scale — a frequency scale approximating human pitch perception, used in speech features.

Micro-average — averaging a metric weighted by the number of examples in each class.

MRR — see mean reciprocal rank.

N-gram — a sequence of n consecutive tokens.

Naive Bayes — a classifier that assumes features are independent given the class.

Normalisation — the decisions that make different strings count as the same text.

Perplexity — the inverse probability of held-out text, normalised per token; roughly the number of choices the model is hesitating between.

Pointwise mutual information — the log ratio of a pair's joint probability to the product of its individual probabilities.

Precision — the fraction of positive decisions that are correct.

Prompt injection — instructions hidden in text the system reads, intended to redirect its behaviour.

Recall — the fraction of real positives that are found.

Retrieval — selecting documents or passages relevant to a query.

Sampling (decoding) — choosing the next token from a distribution, controlled by temperature and related settings.

Sentence segmentation — dividing text into sentences, complicated by abbreviations and decimals.

Smoothing — redistributing probability mass so that unseen events are not impossible.

Stemming — reducing words to a crude common root by removing affixes.

Stop word — a very frequent word, often removed before modelling and often carrying negation.

Subword tokenisation — splitting text into frequent pieces rather than whole words, so that any string can be encoded.

Temperature — a divisor applied to log-probabilities before sampling; low values sharpen, high values flatten.

tf-idf — term frequency weighted by inverse document frequency.

Token — the unit a model consumes: a word, subword or character.

Type — a distinct word form, as opposed to a token instance.

Word error rate — the fraction of reference words that are substituted, deleted or inserted by a recogniser.

Zipf's law — the inverse relation between a word's frequency and its rank.

Sources and Further Reading

The classical material in this volume is stable and well documented; the later chapters describe methods that are changing while this book is being read. The list separates the two.

Jurafsky, D. and Martin, J. H., *Speech and Language Processing*, third edition draft. The standard text, free online, and unusually good at explaining why a method exists before explaining how it works. Chapters on n-grams, smoothing, vector semantics and machine translation correspond directly to Parts One and Two of this volume.

Manning, C. D., Raghavan, P. and Schütze, H., *Introduction to Information Retrieval*, Cambridge University Press, 2008. Free online. The definitive account of inverted indexes, tf-idf and evaluation. Chapter Eight's treatment of BM25 remains the clearest.

Firth, J. R., "A Synopsis of Linguistic Theory, 1930–1955", in *Studies in Linguistic Analysis*, 1957. The origin of the distributional slogan, in a paper that repays reading for its caution as much as its claim.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J., "Distributed Representations of Words and Phrases and their Compositionality", *NeurIPS*, 2013. The word2vec papers, short and readable, and the source of the analogy demonstrations that were later over-interpreted.

Koehn, P., *Statistical Machine Translation*, Cambridge University Press, 2010. Out of date as a description of current systems and still the best explanation of alignment, phrase extraction and why translation is hard.

Papineni, K., Roukos, S., Ward, T. and Zhu, W.-J., "BLEU: a Method for Automatic Evaluation of Machine Translation", *ACL*, 2002. Read the paper, not the leaderboards. The authors were explicit about what the measure does not capture.

Chen, S. F. and Goodman, J., "An Empirical Study of Smoothing Techniques for Language Modeling", *Computer Speech and Language*, 1999. A careful comparison, and the reason Kneser-Ney is still the reference point.

Goodfellow, I., Bengio, Y. and Courville, A., *Deep Learning*, MIT Press, 2016. Chapters 10 and 12 cover sequence models and applications. Free online.

Rabiner, L. and Juang, B.-H., *Fundamentals of Speech Recognition*, Prentice Hall, 1993. Old, and the clearest treatment of the acoustic model, lexicon and

language model division that modern systems absorbed.

Bird, S., Klein, E. and Loper, E., *Natural Language Processing with Python*, O'Reilly, 2009. Free online, dated in its APIs, and still the most practical introduction to counting things in a corpus.

Gebru, T. et al., "Datasheets for Datasets", *Communications of the ACM*, 2021, and **Mitchell, M. et al., "Model Cards for Model Reporting", *FAT***, 2019. Two short papers that formalise the documentation this volume's final chapter argues for.

NIST, *The Text REtrieval Conference (TREC) guidelines*. If you need to build an evaluation set for retrieval, read how TREC has done it for thirty years; the pooling method and the labelling discipline are worth copying.

On the practical side, the documentation of the libraries you use is more useful than most papers. Read the tokeniser documentation carefully: it will tell you how unknown words are handled, how whitespace is treated, and how many tokens your text will actually become — three facts that determine cost, accuracy and comparability, and three facts that are almost never written down in project notes.